DAY 1 — Azure Intro + Concepts + CLI + Linux Foundations

Scott Duffy Sections 1, 2, 3 | 10 Hours | Week 1 of 4

Sections today	Section 1: Course Intro (25 min) | Section 2: Azure Concepts (19 min) | Section 3: PowerShell and CLI (31 min)
Linux focus	Terminal navigation, file operations, man pages, OverTheWire Bandit levels 0–5
Project	WSL2 setup + Azure CLI installed + GitHub repo created + first commit pushed
Goal by end of day	You have Azure free account running, CLI works from WSL2, you understand the full Azure hierarchy, and your GitHub portfolio repo exists.

PART 1 — AZURE CONTENT (4 hours) | Sections 1, 2, 3

Section 1 — Microsoft Azure Administrator Course Intro (25 min)

This section sets the context. Watch it fully at 1x speed because Scott explains how the exam is structured and what Microsoft actually tests. Key things to absorb:

- The AZ-104 exam tests hands-on skill, not memorisation. Microsoft wants to know if you can actually administer Azure.
- The exam has 5 domains: Identity/Governance (20–25%), Storage (15–20%), Compute (20–25%), Networking (15–20%), Monitor/Backup (10–15%).
- Scott's course follows these exact 5 domains in the same order as the exam objectives.
- You will need a free Azure account. Create it at azure.microsoft.com/free — $200 credit for 30 days + always-free services.
- Download the free study guide PDF Scott provides inside the course — print it or keep it open alongside every session.

IMPORTANT	After watching Section 1: Create your Azure free account RIGHT NOW if you have not done it. You cannot do any hands-on work without it. Go to portal.azure.com and log in to confirm access.

Section 2 — Azure Concepts (19 min)

This is the foundation everything else builds on. Every future concept — VNets, storage accounts, VMs — exists within this hierarchy. You must understand it completely.

Azure Region	
What it is	A geographic location containing one or more datacenters. Examples: North Europe (Dublin), West Europe (Amsterdam), East US, Southeast Asia. When you create any resource you choose which region it lives in.
Why it matters	Latency (closer region = faster response for users), compliance (GDPR requires data to stay in EU — use North Europe or West Europe for Danish companies), availability (not all services exist in all regions), cost (prices vary by region).
Real example	*A Danish company hosting their e-commerce platform should choose North Europe (Dublin) or West Europe (Amsterdam) to satisfy GDPR and minimise latency for Danish customers.*

Availability Zone	
What it is	Physically separate datacenters within the same region, each with independent power, cooling, and networking. Typically 3 zones per region. Zonal resources are pinned to a specific zone. Zone-redundant resources spread across all zones automatically.
Why it matters	If one datacenter has a power outage, your VM in Zone 2 keeps running. This is the difference between 99.9% uptime (single VM) and 99.99% uptime (VMs across 2+ zones). Real production systems always use Availability Zones.
Real example	*Your web server VM in Zone 1 goes down due to a cooling failure. Your identical VM in Zone 2 continues serving traffic. The Azure Load Balancer automatically stops sending traffic to Zone 1.*

Azure Geography	
What it is	A group of regions that share the same data residency and compliance boundary. Example: 'Europe' geography contains North Europe and West Europe. Your data never leaves the geography unless you explicitly configure cross-geography replication.

Why it matters	Legal compliance. GDPR requires EU citizen data to stay within the EU geography. If your company stores health data, you may be legally required to keep it within a specific geography.
Real example	*A Danish hospital must store patient data. They choose the Europe geography. Even with geo-redundant storage, replication only happens between North Europe (Dublin) and West Europe (Amsterdam) — never to US regions.*

Resource Group	
What it is	A logical container that holds related Azure resources. A resource group is NOT a billing boundary or a security boundary by itself — it is purely organisational. Resources in a group can be in different regions. A resource can only belong to ONE resource group at a time.
Why it matters	It is the unit for lifecycle management. When you delete a resource group, ALL resources inside it are deleted. This makes it ideal for project environments — delete the group = clean teardown. RBAC roles assigned at resource group scope apply to all resources inside.
Real example	*You create a resource group called 'project-webapp-dev'. Inside: VNet, 2 VMs, storage account, App Service. When development is done, delete the resource group — everything is gone in one action, no resource orphans.*

Azure Subscription	
What it is	A billing and management boundary. Every resource you create belongs to a subscription. A single Azure AD tenant (Entra ID directory) can have multiple subscriptions. Subscriptions have spending limits and quotas. You get billed per subscription.
Why it matters	Cost separation. A company might have one subscription for production, one for development, one for testing. Each gets its own invoice. Cost Management policies and budgets are set at subscription level.
Real example	*Maersk has a subscription called 'Maersk-Production' and 'Maersk-Development'. Engineers can freely test things in the dev subscription without risk of affecting production resources or incurring unexpected production costs.*

Management Group	
What it is	A container for subscriptions. You can apply Azure Policies and RBAC at the management group level and they automatically apply to all subscriptions underneath. The root management group is at the top. You can nest management groups (up to 6 levels deep, max 10,000 management groups).
Why it matters	Enterprise-scale governance. Instead of configuring the same policy on 50 subscriptions individually, apply it once to the management group and all 50 subscriptions inherit it automatically.
Real example	*Maersk has a management group 'Maersk-Corp' with policy: 'All resources must have a CostCenter tag.' This policy flows down to all 47 Maersk subscriptions automatically. No resource can be created without the tag.*

EXAM ALERT	The Azure hierarchy from top to bottom: Management Group > Subscription > Resource Group > Resource. Permissions and policies cascade DOWN this hierarchy. This appears on almost every AZ-104 governance question.

Section 3 — PowerShell and CLI (31 min)

Watch this section fully. Scott shows both Azure PowerShell and Azure CLI. As a Cloud/Infrastructure Engineer targeting Linux environments, Azure CLI (az) will be your primary tool. But know both — some Azure operations only exist in PowerShell and some companies mandate it.

Azure CLI — Essential Commands You Must Know

az login	
What it does	Authenticates your terminal session to Azure using a browser-based OAuth flow. Opens browser, you log in, token is stored locally.
Why use it	Every CLI session starts with this. Without it, no az command will work. Run this first in every new WSL2 session.
Sample output	`To sign in, use a web browser to open https://microsoft.com/devicelogin and enter code XXXXX to authenticate.`

az account show	
What it does	Shows the currently active subscription details — subscription name, ID, tenant ID, and your username.
Why use it	Verify you are working in the correct subscription before creating resources. Creating expensive resources in the wrong subscription is a common mistake.
Sample output	`{ "id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx", "name": "Azure Free Trial", "tenantId": "yyyyyyyy..." }`

az account list --output table	
What it does	Lists all subscriptions your account has access to, in a human-readable table format.
Why use it	If you have access to multiple subscriptions (common in real jobs), this shows all of them so you can switch to the right one.
Sample output	`Name State IsDefault ---------- ------- --------- Azure Free Trial Enabled True`

az account set --subscription <name-or-id>	
What it does	Switches the active subscription for all subsequent CLI commands.
Why use it	When you have multiple subscriptions, always set the correct one before creating resources. Forgetting this = resources in wrong subscription = billing surprise.

az group create --name myRG --location northeurope	
What it does	Creates a resource group named 'myRG' in the North Europe region (Dublin datacenter).
Why use it	Always create a resource group first before any other resource. It is the container for everything. Use North Europe for Danish company compliance.
Sample output	`{ "id": "/subscriptions/.../resourceGroups/myRG", "location": "northeurope", "name": "myRG" }`

az group list --output table	
What it does	Lists all resource groups in the current subscription in a table.
Why use it	Quickly audit what resource groups exist and confirm your group was created. Add --query '[].{Name:name,Location:location}' to filter columns.
Sample output	`Name Location ProvisioningState ------ ---------- ---------- myRG northeurope Succeeded`

az group delete --name myRG --yes --no-wait	
What it does	--yes skips the confirmation prompt. --no-wait returns immediately without waiting for deletion to complete (deletion continues in background).
Why use it	The fastest way to clean up ALL resources in a resource group in one command. In real work you use this constantly to avoid Azure costs from forgotten resources.

Azure PowerShell — Essential Equivalents

`Connect-AzAccount`	
What it does	PowerShell equivalent of az login. Authenticates your session.
Why use it	Some Azure operations only exist in PowerShell (older ARM resources, some legacy features). Teams with Windows background often use PowerShell exclusively.

`Get-AzSubscription`	
What it does	PowerShell equivalent of az account list. Lists all subscriptions.
Why use it	Use in scripts when you need to iterate over subscriptions programmatically.

`New-AzResourceGroup -Name myRG -Location northeurope`	
What it does	PowerShell equivalent of az group create.
Why use it	PowerShell uses verb-noun naming convention. New = create, Get = read, Set = update, Remove = delete. This pattern applies to all Az* commands.

Why Linux matters for a Cloud Engineer

Every Azure VM running production workloads runs Linux (Ubuntu, RHEL, SLES). You will SSH into these VMs to troubleshoot, configure, and automate. The terminal is your primary interface. From Day 1, do everything from the terminal — never use File Explorer.

WSL2 Setup (do this first)

`wsl --install`	
What it does	Installs WSL2 and Ubuntu on Windows 11 in one command. Run in PowerShell as Administrator. Reboot after.
Why use it	WSL2 gives you a full Linux kernel running inside Windows. All your cloud engineering tools (az CLI, Terraform, kubectl, git) run natively in this Linux environment.
Sample output	`Ubuntu is already the default. After reboot, open 'Ubuntu' from Start menu to complete setup.`

`wsl --list --verbose`	
What it does	Shows all installed WSL distributions and their WSL version.
Why use it	Confirm your Ubuntu is running WSL2 (not WSL1). WSL2 has a real Linux kernel — essential for proper networking and Docker support.
Sample output	`NAME STATE VERSION * Ubuntu Running 2`

Terminal Navigation — Complete Reference

These are the commands you will use every single day as a cloud engineer. Learn them until they are automatic — you should not need to think about basic navigation.

`pwd`	
What it does	Print Working Directory. Shows the full path of the directory you are currently in.
Why use it	Never lose track of where you are. Before running any command that creates or deletes files, run pwd first to confirm your location. Deleting files in the wrong directory is permanent.
Sample output	`/home/yourname/projects/azure`

`ls -la`	
What it does	-l = long format (shows permissions, owner, size, date). -a = all files including hidden ones (files starting with dot like .ssh, .gitignore).
Why use it	ls alone shows files. ls -la shows EVERYTHING including hidden files and their permissions. When debugging configuration issues, hidden files (.env, .ssh/config) are often the cause — ls -la reveals them.
Sample output	`drwxr-xr-x 2 user user 4096 Apr 1 09:00 . drwxr-xr-x 15 user user 4096 Apr 1 08:00 .. -rw-r--r-- 1 user user 220 Apr 1 08:00 .bash_logout`

`cd ~`	
What it does	Change directory to your home directory (/home/username). ~ is a shortcut for home.

Why use it	When lost in the filesystem, cd ~ gets you back to safety. Home is where your .ssh keys, .bashrc, and config files live.

`mkdir -p projects/azure/week1`	
What it does	-p = create parent directories as needed. Without -p, mkdir fails if 'projects' or 'azure' don't exist yet.
Why use it	Create nested directory structures in one command. Use this daily to organise your work. -p makes scripts safe — mkdir -p never fails if directory already exists.

`cp -r source/ destination/`	
What it does	-r = recursive. Copies a directory and ALL its contents recursively.
Why use it	Without -r, cp fails on directories. Always use -r when copying folders. In Azure work: copying config directories, backup configs before editing.

`mv oldname newname`	
What it does	Move (rename) a file or directory. If destination is in same directory = rename. If different directory = move.
Why use it	mv is also how you rename files in Linux. There is no separate 'rename' command. Used constantly: renaming scripts, moving configs, organising project files.

`rm -rf directory/`	
What it does	-r = recursive (delete directory and all contents). -f = force (no confirmation prompts, ignores missing files).
Why use it	The nuclear option. Instantly and permanently deletes everything. NO UNDO. Always double-check the path with pwd and ls before running. Never run rm -rf / or rm -rf * from the wrong directory.

IMPORTANT	NEVER run: rm -rf / or rm -rf ~ or rm -rf . without knowing exactly what you are doing. These delete your entire system, home directory, or current directory. Always verify your location with pwd before using rm -rf.

`cat filename`	
What it does	Display the entire content of a file in the terminal.
Why use it	Quick file inspection. View config files, logs, scripts. For large files use less instead (it paginates). cat is for short files — less than 50 lines.

`less filename`	
What it does	Display file content with pagination. Navigate: Space = next page, b = previous page, /search = search, q = quit.
Why use it	Use for large files (logs, configs). Unlike cat, less does not load the entire file into memory — essential for gigabyte log files on production servers.

`grep -r 'pattern' /path/`	
What it does	-r = recursive search through directories. Finds all files containing the pattern.
Why use it	Find specific text across thousands of files instantly. Essential for: finding where a config value is set, searching logs for error messages, finding which script uses a variable.
Sample output	`grep -r 'northeurope' /home/user/terraform/  — finds all .tf files mentioning northeurope`

`grep -i 'error' /var/log/syslog | tail -20`	
What it does	Pipes grep (find lines with 'error', -i = case insensitive) into tail (show last 20 lines).
Why use it	The pipe | is fundamental. It sends the output of one command as input to another. This combination is used constantly for log analysis — find errors in logs and show the most recent ones.

`man ls`	
What it does	Opens the manual page for a command. q to quit. / to search within man page.
Why use it	The most important command to know when you forget how something works. Every command has a man page with all options documented. 'man man' shows the manual for the manual itself.

PRO TIP	When you encounter an unfamiliar command or flag in your career: man <command> is always the right first step. It is faster than Google and gives you the definitive answer. Cloud engineers who read man pages solve problems faster.

OverTheWire Bandit — Levels 0 to 5

Go to overthewire.org/wargames/bandit. These challenges teach Linux by forcing you to use it to solve puzzles. Each level teaches a specific skill you will use as a cloud engineer.

Level 0	SSH into the server: ssh bandit0@bandit.labs.overthewire.org -p 2220. Password: bandit0. WHY: this is exactly how you connect to Azure VMs. SSH is your primary interface to every Linux VM you will ever manage.
Level 0→1	Find password in file 'readme' in home directory. Commands: ls, cat readme. WHY: basic file reading — config files, credentials (in dev), logs are all read this way.
Level 1→2	File named '-' (dash). Use: cat ./- WHY: filenames starting with - are interpreted as flags. Real-world: accidental files with special names. Solution: ./ prefix.
Level 2→3	File with spaces in name. Use: cat 'spaces in this filename'. WHY: file paths with spaces need quoting. Common in scripts and Azure resource names.
Level 3→4	Hidden file in a directory. Use: ls -la inhere/ WHY: hidden config files (.ssh, .env, .gitignore) are invisible without -a flag. Critical for security and debugging.
Level 4→5	Find human-readable file. Use: file inhere/* WHY: file command identifies file types without extensions. On servers you often get files without extensions.

EXAM QUESTIONS — AZ-104 Style

[EXAM Q] A company needs to apply the same Azure Policy to 50 subscriptions. What is the most efficient way to do this?

Answer: Place all 50 subscriptions under a Management Group and assign the Azure Policy to the Management Group. Policies assigned to a management group automatically apply to all subscriptions and resource groups underneath it. This is infinitely more efficient than assigning the policy to each subscription individually.

[EXAM Q] An administrator creates a resource group in West Europe. They then create a VM and specify East US as the location. Where does the VM run?

Answer: The VM runs in East US. The resource group location only determines where the resource group METADATA is stored, not where the resources inside it run. Resources in a resource group can be in any region. This is a common AZ-104 trick question.

[EXAM Q] Your organisation has a subscription called 'Development'. A developer accidentally creates a VM in the Production subscription. What should you implement to prevent this from recurring?

Answer: Implement Azure Policy at the subscription or management group level with a 'deny' effect that restricts resource creation based on subscription tags or naming conventions. Alternatively, use Azure RBAC to restrict the developer's Contributor role to only the Development subscription, not the Production subscription.

[EXAM Q] A company has 3 Azure subscriptions. They want to ensure a specific Azure Policy applies to all current and future subscriptions automatically. What should they use?

Answer: Create a Management Group that contains all 3 subscriptions, and assign the Azure Policy to the Management Group. Any new subscription added to the management group will automatically inherit the policy. This is the primary reason management groups exist.

[EXAM Q] What is the maximum number of management group levels you can have below the root management group?

Answer: Six (6) levels of nesting below the root management group, for a total depth of 7 levels including root. The maximum number of management groups in a single directory is 10,000.

INTERVIEW QUESTIONS — Cloud Engineer Roles in Denmark

[INTERVIEW Q] Explain the Azure resource hierarchy from top to bottom and why it matters.

Answer: The hierarchy is: Root Management Group > Management Groups > Subscriptions > Resource Groups > Resources. It matters because Azure Policy and RBAC permissions cascade DOWN this hierarchy. A policy assigned at management group level automatically applies to all subscriptions, resource groups, and resources below it. This hierarchy is how enterprises like Maersk or Novo Nordisk govern hundreds of subscriptions and thousands of resources consistently.

[INTERVIEW Q] Why would you choose North Europe or West Europe when creating Azure resources for a Danish company?

Answer: Three reasons: 1) GDPR compliance — EU citizen data must remain within the EU and these regions satisfy that requirement. 2) Latency — both regions are geographically close to Denmark (Dublin and Amsterdam), minimising response times for Danish users. 3) Disaster recovery — Azure geo-redundant storage paired between North Europe and West Europe means failover stays within the EU geography, maintaining compliance even during regional outages.

[INTERVIEW Q] What is the difference between a resource group and a subscription in Azure?

Answer: A subscription is primarily a billing and management boundary — each subscription gets its own invoice and has its own resource quotas. A resource group is an organisational container within a subscription for

grouping related resources together for lifecycle management. You can have multiple resource groups per subscription. Deleting a resource group deletes all resources inside it, making it the unit for environment teardown. RBAC and policies applied at subscription level apply to all resource groups in it.

[INTERVIEW Q] How would you quickly delete all resources from a development environment in Azure?

Answer: If all development resources are in a single resource group (which is best practice): az group delete --name dev-rg --yes --no-wait. This deletes the resource group and ALL resources inside it in one command. --yes skips confirmation. --no-wait returns the terminal immediately while deletion happens in background. This is why grouping related resources in a dedicated resource group is important — it enables clean teardown.

PROJECT — Day 1 | GitHub Repo + Azure CLI + WSL2 Verified

This project takes 1 hour. By the end you have a working Azure CLI session in WSL2 and a GitHub portfolio repository with your first commit. Every future project builds on this foundation.

1 Install and configure WSL2 with Ubuntu

CMD

```
# Run this in PowerShell as Administrator:
wsl --install
# After reboot, open Ubuntu and set username/password
wsl --list --verbose   # Confirm VERSION is 2
```

VERIFY You see Ubuntu listed with STATE=Running and VERSION=2

2 Install Azure CLI inside WSL2

CMD

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
az version
```

VERIFY You see JSON output with azure-cli version 2.x.x

3 Authenticate to Azure

CMD

```
az login
# Browser opens — log in with your Azure account
az account show
az account show --output table
```

VERIFY Your subscription name and ID are displayed. State shows Enabled.

4 Explore Azure CLI commands

CMD

```
az group list --output table
az account list-locations --output table | grep -i europe
az group create --name day1-practice-rg --location northeurope
az group show --name day1-practice-rg
az group delete --name day1-practice-rg --yes
```

VERIFY Resource group created then deleted. You are comfortable creating and deleting resource groups via CLI.

5 Install git and configure identity

CMD

```
sudo apt update && sudo apt install git -y
git config --global user.name 'Your Name'
git config --global user.email 'your@email.com'
git config --list
```

VERIFY git config --list shows your name and email correctly.

6 Create GitHub repository and first commit

CMD

```
# On github.com: create repo 'azure-cloud-engineer-dk' (public, with README)
mkdir -p ~/projects && cd ~/projects
git clone https://github.com/yourusername/azure-cloud-engineer-dk
cd azure-cloud-engineer-dk
mkdir -p week1/day1
cat > week1/day1/notes.md << 'EOF'
# Day 1 Notes
## Azure Hierarchy
- Management Group > Subscription > Resource Group > Resource
## CLI commands practiced today
- az login, az account show, az group create/delete
```

```
EOF
git add .
git commit -m 'Day 1: Azure CLI setup and first notes'
git push origin main
```

VERIFY Visit your GitHub repo in browser — you see your first commit with notes.md.

7 Verify your environment is production-ready

CMD

```
# All of these should work:
az account show --query name -o tsv
git status
pwd
ls -la ~/projects/azure-cloud-engineer-dk/
code .   # Opens VS Code with WSL integration if VS Code is installed
```

VERIFY All commands work. No errors. You have a working cloud engineering environment in WSL2.

PRO TIP Install VS Code on Windows and install the 'WSL' extension (Remote - WSL). Then from any WSL2 directory: run 'code .' to open VS Code with full WSL2 integration. This is the professional setup for all Terraform, bash, and YAML editing.

END OF DAY 1 CHECKLIST — Tick everything before you sleep

☐	Azure free account created and accessible at portal.azure.com
☐	WSL2 Ubuntu installed and running (VERSION=2 confirmed)
☐	Azure CLI installed in WSL2 — az version works
☐	az login completed — az account show shows your subscription
☐	Resource group created and deleted via CLI (az group create + az group delete)
☐	Git configured with your name and email
☐	GitHub repo 'azure-cloud-engineer-dk' created
☐	First commit pushed to GitHub — visible in browser
☐	OverTheWire Bandit levels 0–5 completed
☐	Can explain the Azure hierarchy (Management Group > Subscription > Resource Group > Resource) without looking at notes
☐	Can answer all 4 exam questions in this document without looking at answers

TIP	Tomorrow is Day 2 — Entra ID, RBAC, and deploy your first VM. Keep your Azure portal open and your WSL2 terminal ready. Do not close your az login session.